EE5203 L05 Lab Guide

GPIO with the HAL — same hardware, new interface - Basri Erdogan

Mission: Rebuild the L04 LED control with HAL functions, make B1 control LD2 by polling, and prove in the register view that HAL wrote the same bits you wrote by hand last week.

Prepare before class

- ☐ Re-read the theory slides up to "The polling loop."
- ☐ Know last week's three registers: `AHB1ENR`, `MODER`, `BSRR`.
- ☐ Know why B1 released reads `GPIO_PIN_SET` (external pull-up, active-low).

Learning objectives

By checkoff time, you can:

1. configure PA5 (output) and PC13 (input, no pull) in CubeMX and read the generated `MX_GPIO_Init()`;
2. drive and read pins with `HAL_GPIO_WritePin` / `TogglePin` / `ReadPin`;
3. verify a HAL configuration in the SFR view down to the register bits;
4. build a polled button behavior without any direct register writes.

Hardware and files

Item	Required value
Board	Nucleo-F401RE (LD2 on PA5, B1 on PC13 — no wiring)
Input	B1, external pull-up: released = `SET`, pressed = `RESET`
Project	New CubeMX project `L05_HAL_GPIO` (Toolchain = CMake), opened in VS Code
Clock	Default HSI is sufficient

Configure in CubeMX

1. New STM32 project → Board Selector → **NUCLEO-F401RE**; accept default peripheral initialization.
2. `PA5` → `GPIO_Output` (defaults: push-pull, no pull, low speed).
3. `PC13` → `GPIO_Input`, **no pull-up and no pull-down** — the board's external resistor already holds PC13 high.
4. Generate code. CubeMX already calls `MX_GPIO_Init()` in `main()` before the loop — **verify it is there; do not add a second call.**

Checkpoint A - HAL blink and register proof

Inside the `while (1)` user-code region:

```
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
HAL_Delay(500);
```

Build, flash, and confirm the blink. Then pause in the debugger after `MX_GPIO_Init()` and check the SFR view:

Register	Expected
`GPIOA->MODER` bits 11:10	`01` (output)
`GPIOC->MODER` bits 27:26	`00` (input)
`GPIOC->PUPDR` bits 27:26	`00` (no internal pull)

Evidence for Checkpoint A: the three register fields above, plus the blink. HAL wrote what you wrote by hand in L04.

Checkpoint B - Button override

Replace the loop body: blink while B1 is released, hold LD2 **off** while B1 is pressed.

```
GPIO_PinState button = HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13);

if (button == GPIO_PIN_SET) {          /* released */
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    HAL_Delay(500);
} else {                               /* pressed  */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
}
```

Evidence for Checkpoint B: LED dark while B1 is held, blinking resumes on release; watch `GPIOC->IDR` bit 13 change in the SFR view as you press.

Checkpoint C - Press counter with patterns

Count presses and cycle four LED behaviors (off → steady → slow blink → double flash). Detect the press **edge**, not the level:

```
static uint32_t press_count = 0U;
static uint32_t prev = 1U;                      /* released */

uint32_t cur = (uint32_t)HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13);
if (prev == 1U && cur == 0U) {                  /* falling edge */
    ++press_count;
    HAL_Delay(10);                              /* debounce sample */
}
prev = cur;

switch (press_count & 0x3U) { /* four cases: your patterns here */ }
```

Evidence for Checkpoint C: four presses walk through all four patterns and return to the first.

Individual extension - Bare-metal rematch

Recreate Checkpoint B **without HAL calls**: clocks via `RCC->AHB1ENR` (GPIOA bit 0, GPIOC bit 2), PA5 output via `MODER`, read `GPIOC->IDR` bit 13, drive PA5 via `BSRR`. Leave PC13 with **no internal pull** — the external resistor is already there. Keep it in a separate source file or `#ifdef` block.

Acceptance checklist

- ☐ `.ioc`: PA5 output, PC13 input with no pull; generated code untouched outside `USER CODE` regions.
- ☐ Register proof for all three fields in Checkpoint A.
- ☐ Button override works and `IDR` bit 13 evidence shown.
- ☐ Press counter cycles four patterns with edge detection.
- ☐ Extension demonstrated in one interface only (no HAL/register mixing).

Individual oral check

- Which register bits did `GPIO_MODE_OUTPUT_PP` change, and to what?
- Why must PC13 use `GPIO_NOPULL` on this board?
- Why does the press counter compare `prev` and `cur` instead of testing `cur == 0` alone?
- What does a successful build *not* prove?

Submit

Submit `main.c` (and the bare-metal file if attempted) plus one SFR-view screenshot showing the three Checkpoint A register fields.

If you are stuck

Check in order: `MX_GPIO_Init()` runs before the loop (once); the `.ioc` pin settings match this card; the SFR view shows the expected `MODER`/`PUPDR` fields; `IDR` bit 13 follows the physical button; only then suspect your loop logic. **LED inverted?** You swapped `SET`/`RESET` meaning. **Count jumps by 2–3 per press?** Your edge test or debounce delay is missing.